Vision-Based Tetris Advisor

A Vision System for Automated Move Recommendation

Digital Image Processing (DIP)

Semester Project Report

Group Members

Akif Jawad | *23i-0583*

Faran Azam | *23i-0075*

Rehan Akram | *23i-0500*

Date: May 2025

Abstract

This report presents a vision-based Tetris advisory system developed as a semester project for a Digital Image Processing course. The system accepts a static Tetris gameplay screenshot as input and produces an annotated output image indicating the optimal rotation and drop column for the currently active piece. The pipeline comprises seven sequential stages: image preprocessing (grayscale conversion, Gaussian blur, and Otsu binarization), board detection via perspective transformation, grid parsing using HSV color-space masking and cell fill-ratio analysis, piece detection through connected-component analysis and Hu moment classification, move evaluation using the Dellacherie six-feature scoring function with two-piece lookahead, and visual output rendering via weighted image blending and OpenCV drawing primitives. On a labeled validation set of 107 training images spanning all seven standard Tetris tetrominoes (I, O, J, L, S, T, Z), the piece classification sub-system achieved 100% accuracy with zero misclassifications. The full pipeline runs in approximately 100–300 ms per frame, comfortably below the 500 ms target. All image processing was implemented using OpenCV 4.x and NumPy; no machine learning was employed. Results demonstrate that classical image processing techniques are fully sufficient for robust Tetris board understanding under controlled screenshot conditions.

1. Introduction

1.1 Problem Statement

Tetris is one of the most well-studied games in artificial intelligence and combinatorial optimization research, yet human players must make sub-second spatial reasoning decisions under significant cognitive load. Determining the optimal placement for a falling piece requires simultaneously evaluating board topology, piece geometry, rotation, lateral position, and the identity of the upcoming piece — a task that exhausts working memory at higher speeds. A computer vision system that can perceive the board state directly from a screenshot and compute the mathematically optimal move provides both a practical playing aid and a compelling demonstration of classical image processing techniques applied to a real-world problem.

1.2 Project Goals

The primary goal of this project is to construct an end-to-end pipeline that: (1) accepts an arbitrary Tetris gameplay screenshot as input; (2) autonomously extracts the current board state, the identity of the active piece, and the identity of the next-piece preview; (3) computes the optimal rotation and column for the active piece using a proven evaluation function; and (4) produces an annotated output image with a confidence heatmap. A secondary goal is to validate the piece-detection sub-system rigorously against a labeled ground-truth dataset. An optional stretch goal of frame-by-frame video processing was scoped but not the primary deliverable.

1.3 Scope and Approach

The project deliberately restricts itself to classical image processing — no convolutional neural networks or learned feature detectors are used. All perception is achieved through deterministic algorithms: color-space transformations, morphological operations, geometric transforms, connected-component analysis, and moment-based shape descriptors. The AI component (move selection) likewise uses a deterministic, analytically derived evaluation function (Dellacherie, 2003) rather than reinforcement

learning. This scope aligns with the course objectives of demonstrating deep fluency with OpenCV's core API and the theoretical foundations of digital image processing.

1.4 Tools and Environment

The implementation is written entirely in Python 3.9+, using OpenCV 4.9.0 for image processing primitives, NumPy 1.26.4 for array manipulation, and Matplotlib 3.8.4 for supplementary visualization. All code is organized into a modular package under *src/*, with a single entry-point script *main.py* and supporting scripts for batch processing and validation.

2. System Pipeline

The system processes a single screenshot through a strictly sequential seven-stage pipeline. Each stage is implemented as a standalone Python module, accepting well-defined NumPy array inputs and producing typed outputs, which facilitates isolated unit testing and the optional *--debug* mode that writes intermediate images to disk.

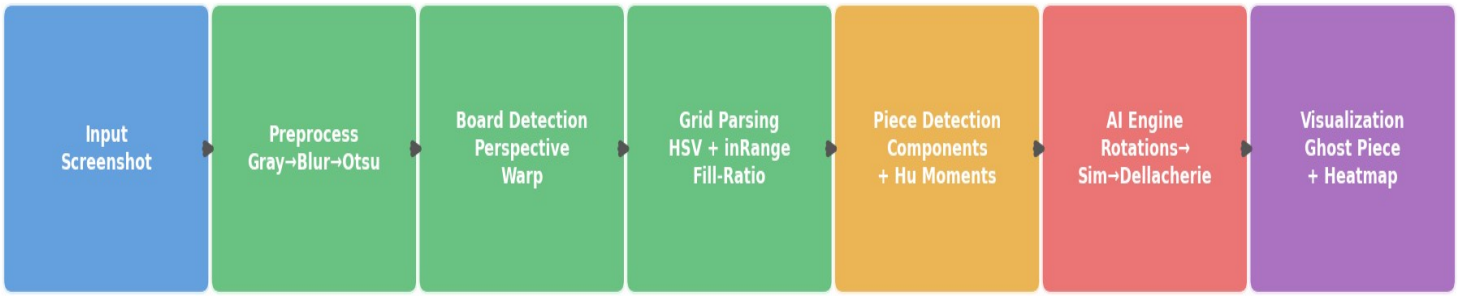

Figure 1: Seven-stage pipeline block diagram. Each box corresponds to one Python module in src/.

Stage 1 — Preprocess (src/preprocess.py)

The raw screenshot is loaded in BGR color space and converted to grayscale using **cv2.cvtColor**. A 5×5 Gaussian blur kernel is applied to suppress high-frequency sensor noise and JPEG compression artifacts before binarization. Otsu's method then automatically selects the globally optimal threshold, yielding a clean binary image that isolates foreground elements from the background without any manually tuned threshold constants.

Stage 2 — Board Detection (src/board_detector.py)

The playfield region is isolated using a perspective warp. Configured corner coordinates (stored in *config.py*) define the four corners of the game board in screen-pixel space. **cv2.getPerspectiveTransform** computes a 3×3 homography matrix mapping these corners to a canonical upright rectangle, and **cv2.warpPerspective** applies the warp.

The same procedure extracts the next-piece preview region independently. This stage eliminates perspective skew and normalizes the board to a fixed pixel grid for all downstream processing.

Stage 3 — Grid Parsing (src/grid_parser.py)

The warped board image is re-converted to HSV color space to separate chromatic content from luminance. Two color masks are constructed with **cv2.inRange**: one targeting chromatic (colored) Tetris blocks at high saturation, and one targeting achromatic (gray/white) blocks such as ghost pieces at low saturation and high value. The masks are combined via **cv2.bitwise_or** to form a unified block mask. The board is then divided into a 20×10 grid, and each cell is classified as filled or empty based on whether ≥50% of its interior pixels lie within the mask.

Stage 4 — Piece Detection (src/piece_detector.py)

Connected-component analysis via a custom 4-directional BFS traversal identifies contiguous groups of filled cells. Groups of exactly four cells are matched against a pre-defined tetromino pattern database (primary classifier). For ambiguous or merged groups, a secondary classifier extracts the top-most four-cell subset using DFS. If pattern matching fails, Hu moments (**cv2.moments → cv2.HuMoments**) provide a rotation- and scale-invariant fallback that compares the shape against seven pre-computed reference descriptors. The same logic runs on the warped next-piece preview region.

Stage 5 — AI Engine (src/engine.py)

All unique rotations of the active piece are generated via **np.rot90** and deduplicated. For each (rotation, column) combination, the simulator drops the piece using gravity with collision detection, clears completed lines, and returns the resulting board state. Each resulting board is scored with the Dellacherie six-feature evaluation function. A two-piece lookahead then evaluates all follow-up placements of the next piece for each active-piece candidate, and the active move that maximises the best achievable follow-up score is selected.

Stage 6 — Visualization (src/visualizer.py)

The recommended move is rendered onto the original screenshot using **cv2.fillConvexPoly** for the semi-transparent ghost piece, **cv2.rectangle** for the highlighted column, and **cv2.putText** for the info panel. A per-column confidence heatmap is generated by normalizing Dellacherie scores to [0, 1] and mapping them to a Red→Yellow→Green palette before blending at α = 0.4 with **cv2.addWeighted**.

Stage 7 — Output

The annotated image and the heatmap image are saved to *output/* and *output/heatmap/* respectively. When the *--debug* flag is passed, all ten intermediate DIP debug images (grayscale, blurred, binary, warped board, warped next-piece region, grid mask, grid overlay, active piece highlight, next piece detection, and region bounding boxes) are also written to *output/debug/*.

3. Image Processing Techniques

This section provides a detailed account of each image processing technique employed in the pipeline, covering the theoretical basis of the technique, the specific manner in which it was applied, and the rationale for choosing it over plausible alternatives.

3.1 Grayscale Conversion

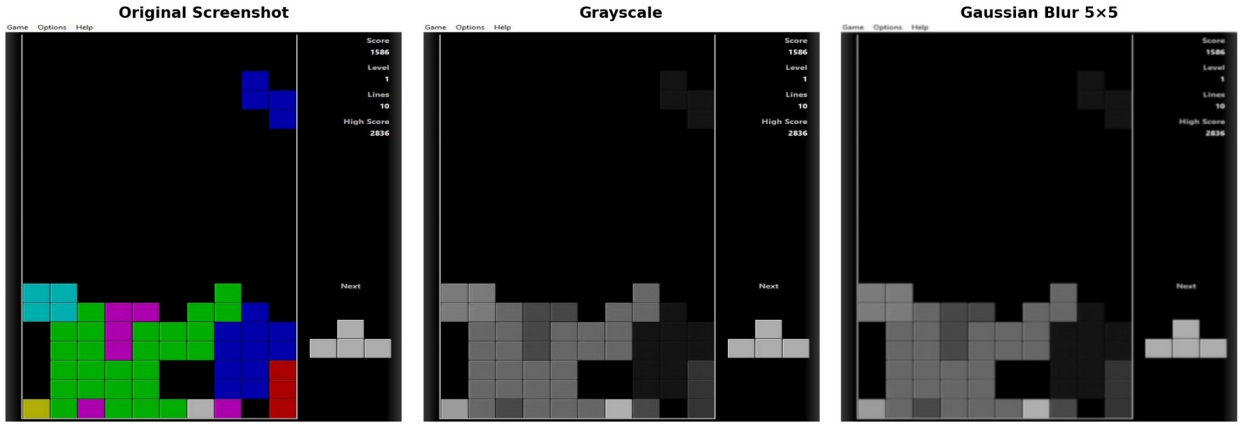

Figure 2: Left: original BGR screenshot. Centre: after grayscale conversion (cv2.cvtColor, BGR→GRAY). Right: after Gaussian blur (5×5 kernel). Note the progressive reduction of color and high-frequency detail.

Grayscale conversion collapses a three-channel BGR image into a single luminance channel using the standard perceptual weighting formula:

$$Y = 0.114 \cdot B + 0.587 \cdot G + 0.299 \cdot R$$

OpenCV's **cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)** applies exactly this weighting. Reducing the image to a single channel eliminates irrelevant chromatic variability that could cause the same board state to have different pixel values across different Tetris themes or monitor color profiles. It also reduces the memory footprint and compute cost of all subsequent convolutions by a factor of three. Alternative approaches such as operating in a single BGR channel (e.g., only green) would introduce a directional bias and fail for non-standard color themes.

3.2 Gaussian Blur

Gaussian blur applies a separable 2D convolution using a kernel whose weights are sampled from a bivariate Gaussian distribution:

$$G(x, y) = (1 / 2\pi\sigma^2) \cdot exp(-(x^2 + y^2) / 2\sigma^2)$$

We apply a 5×5 kernel, which corresponds roughly to $\sigma \approx 1.0$. This kernel size suppresses individual-pixel noise and JPEG compression block artifacts without eliminating the coarse structure of cell boundaries. The blur is applied before thresholding to prevent high-frequency noise from creating spurious binary regions. A uniform (box) blur would similarly reduce noise but produces ringing artifacts at edges; Gaussian blur is the preferred choice because it has no sidelobes in the frequency domain and preserves edge gradients more faithfully.

3.3 Otsu Thresholding

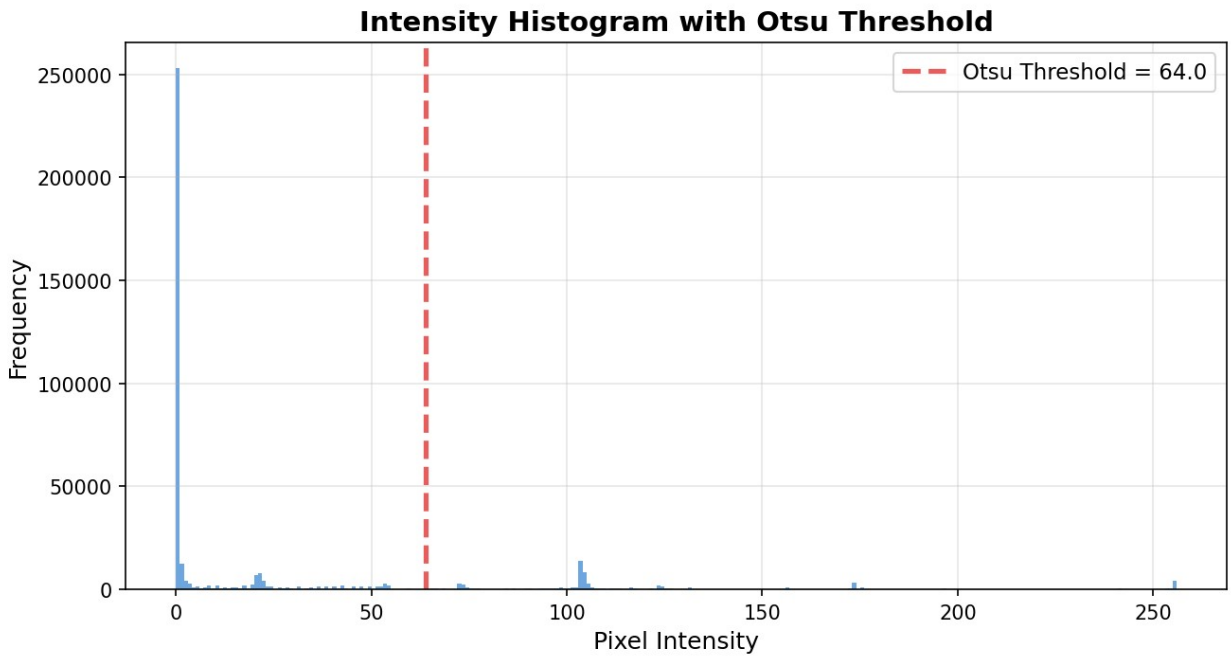

Figure 3: Pixel intensity histogram of a representative screenshot after Gaussian blur. The dashed vertical line marks the automatically selected Otsu threshold, which sits in the valley between the background and foreground modes.

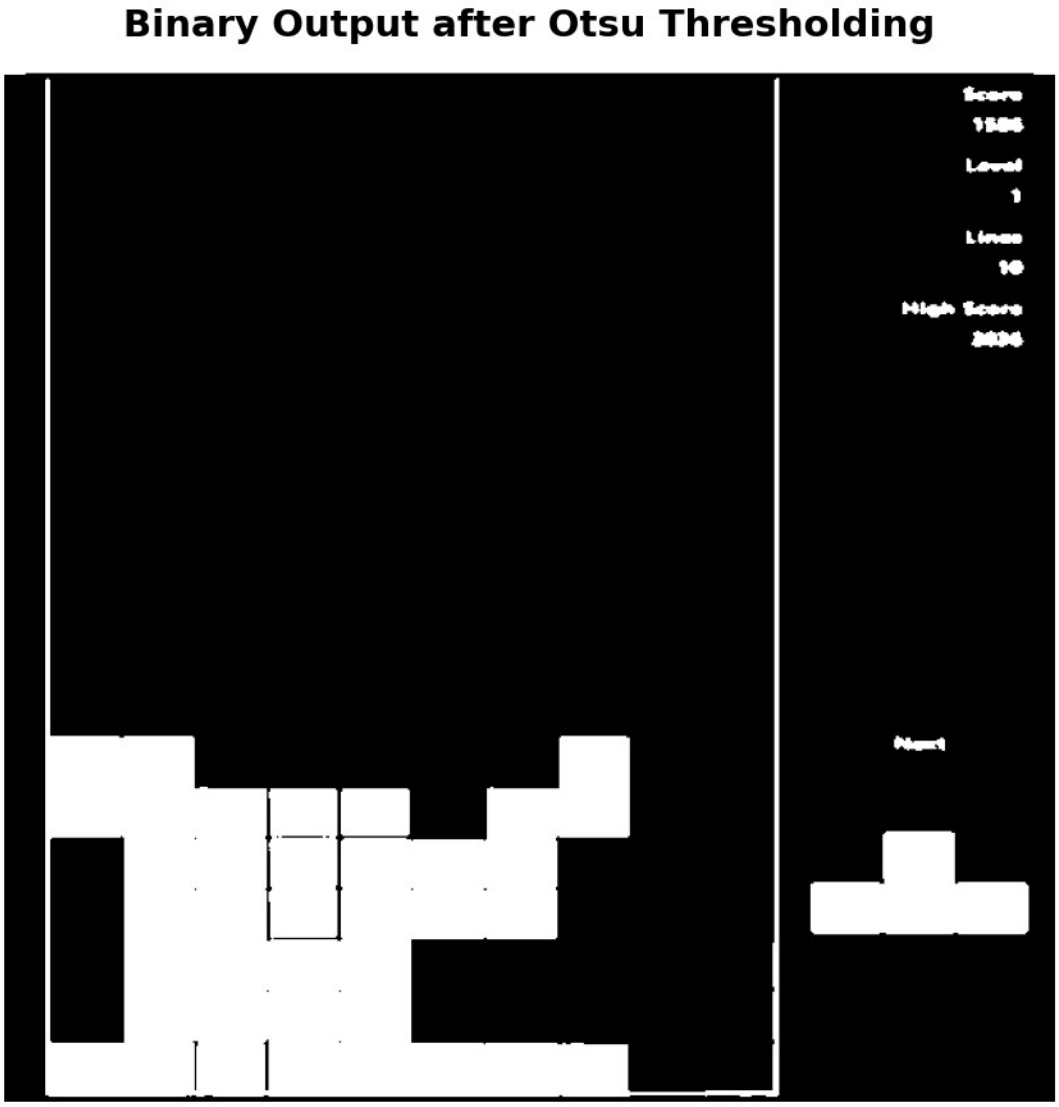

Figure 4: Binary output produced by Otsu thresholding. Foreground structures (board border, piece cells, grid lines) are rendered white; background is black.

Otsu's method (Otsu, 1979) selects the threshold T* that maximises the between-class variance of the binarized image:

$$T^* = \arg\max_T \left[\omega_0(T)\cdot\omega_1(T)\cdot(\mu_0(T)-\mu_1(T))^2 \right]$$

where ω_0, ω_1 are the fractional pixel counts and μ_0, μ_1 are the mean intensities of the background and foreground classes at threshold T. In **cv2.threshold** with the

THRESH_OTSU flag, OpenCV scans all 256 possible thresholds, evaluates this objective, and returns T*. The key advantage over a fixed threshold is full adaptivity to ambient lighting conditions and game-specific color palettes: no manual recalibration is required when switching between Tetris implementations. A Canny edge detector was considered as an alternative for board detection, but Otsu binarization is simpler and provides a filled mask rather than just edges, which is more useful for downstream cell fill-ratio analysis.

3.4 HSV Color Space and Color Thresholding (cv2.inRange)

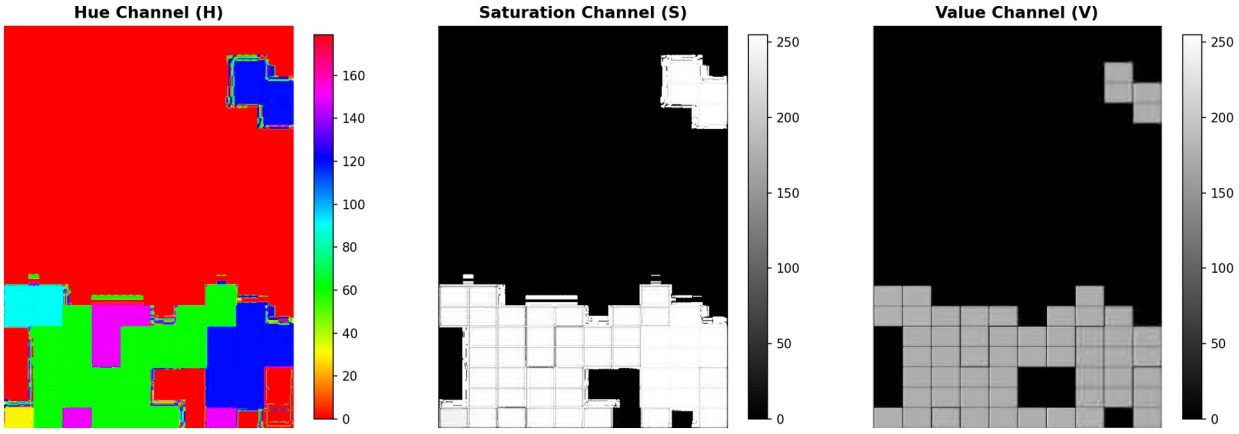

Figure 5: The warped board image decomposed into its three HSV channels. Left: Hue channel (piece color identity). Centre: Saturation channel (separates colored pieces from gray/white). Right: Value channel (luminance, useful for detecting bright ghost pieces).

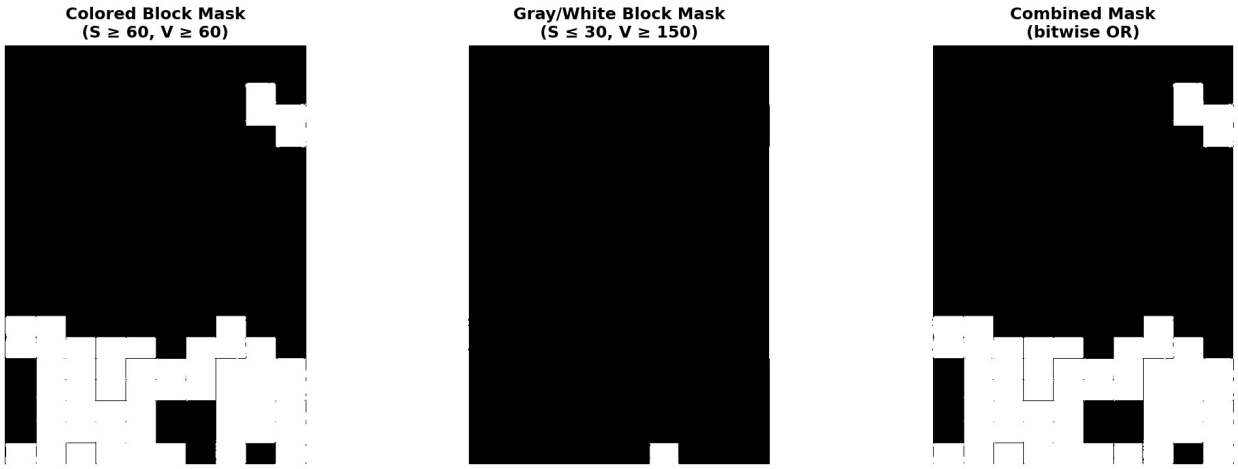

Figure 6: Color segmentation masks. Left: colored-block mask (high-saturation inRange). Centre: gray/white block mask (low saturation, high value). Right: combined mask after cv2.bitwise_or.

Although Otsu binarization is used during board detection, grid parsing requires color-aware segmentation to correctly classify cells at the grid-parsing stage. The warped board image is re-converted to HSV color space using **cv2.cvtColor(board, cv2.COLOR_BGR2HSV)**. HSV separates chromatic information (Hue, 0–180° in OpenCV) from achromatic information (Saturation, Value), making it far more robust to brightness variation than RGB-based thresholding. Two complementary masks are then computed with **cv2.inRange**:

Colored block mask: H ∈ [0, 180], S ∈ [60, 255], V ∈ [60, 255] — captures all saturated Tetris piece colors (cyan, blue, orange, yellow, green, purple, red).

Gray/white block mask: S ∈ [0, 30], V ∈ [150, 255] — captures ghost pieces and any desaturated foreground elements.

BGR-space thresholding was considered but rejected because the same color appears at very different (R, G, B) triplets depending on screen gamma and ambient light; the Hue component in HSV is far more invariant across these variations.

3.5 Bitwise Operations (OR)

The two HSV masks produced in Section 3.4 are merged using **cv2.bitwise_or(colored_mask, gray_mask)**. A bitwise OR at each pixel position returns 255 if either mask is active at that pixel and 0 otherwise, forming a unified block mask that captures all foreground cell types simultaneously. This is analytically equivalent to a set union over the two color classes. An addition operation would serve the same purpose mathematically but risks pixel saturation artifacts; bitwise OR is exact for binary masks. The unified mask is then used directly in the cell fill-ratio computation.

3.6 Perspective Transform

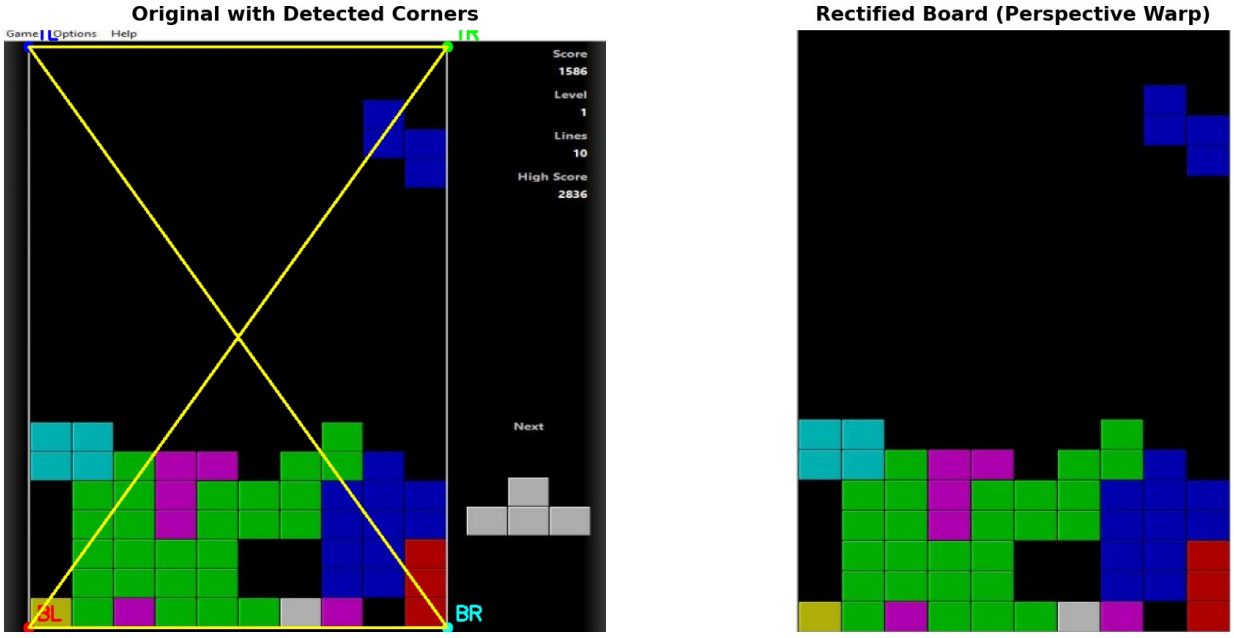

Figure 7: Left: original screenshot with the four board corner coordinates highlighted. Right: rectified board region after cv2.warpPerspective, normalized to a fixed rectangular grid.

Perspective distortion arises when the camera or display is not perfectly orthogonal to the capture plane. **cv2.getPerspectiveTransform** computes the 3×3 projective homography matrix H mapping four source points (board corners in screen space) to four destination points (corners of a canonical rectangle) by solving the system:

$$s \cdot [x', y', 1]^T = H \cdot [x, y, 1]^T$$

where (x, y) are source pixel coordinates, (x', y') are destination pixel coordinates, and s is a projective scale factor. **cv2.warpPerspective** then applies inverse mapping with bilinear interpolation to fill the destination image, ensuring that every cell in the 20×10 grid maps to a uniform rectangular region in the warped image. This is critical for the subsequent fill-ratio analysis, which assumes uniform cell dimensions. A simple affine warp (three-point) would be insufficient for general perspective correction, though it is an acceptable approximation when the board is viewed nearly head-on.

3.7 Morphological Operations

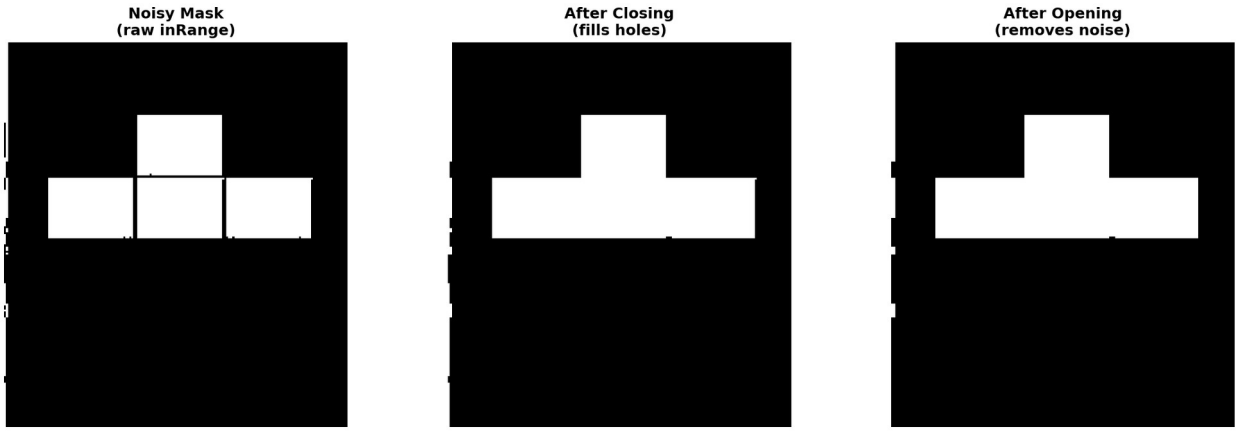

Figure 8: Effect of morphological operations on the next-piece preview mask. Left: raw HSV mask with small holes and noise. Centre: after closing (dilation then erosion) — holes inside blocks filled. Right: after opening (erosion then dilation) — isolated noise pixels removed.

Morphological operations manipulate binary images using a structuring element (kernel). Two compound operations are applied to the next-piece mask using **cv2.morphologyEx**:

Closing (cv2.MORPH_CLOSE): Dilation followed by erosion. Dilation grows foreground regions by the kernel radius, filling small holes and thin gaps; subsequent erosion restores the original boundary, leaving only the filled holes. This corrects the partial shading or anti-aliasing that sometimes leaves interior pixels of a piece cell unmasked.

Opening (cv2.MORPH_OPEN): Erosion followed by dilation. Erosion removes small isolated foreground blobs; dilation then restores objects large enough to survive the

erosion. This eliminates speckle noise introduced by JPEG compression artifacts or thin grid lines that pass the HSV threshold.

The structuring element is a 3×3 rectangular kernel in both cases. Morphological operations are only applied to the next-piece preview mask (a small ≈170×190 px region); the main board mask relies on cell fill-ratio thresholding, which is inherently more robust to small within-cell noise.

3.8 Cell Fill-Ratio Analysis

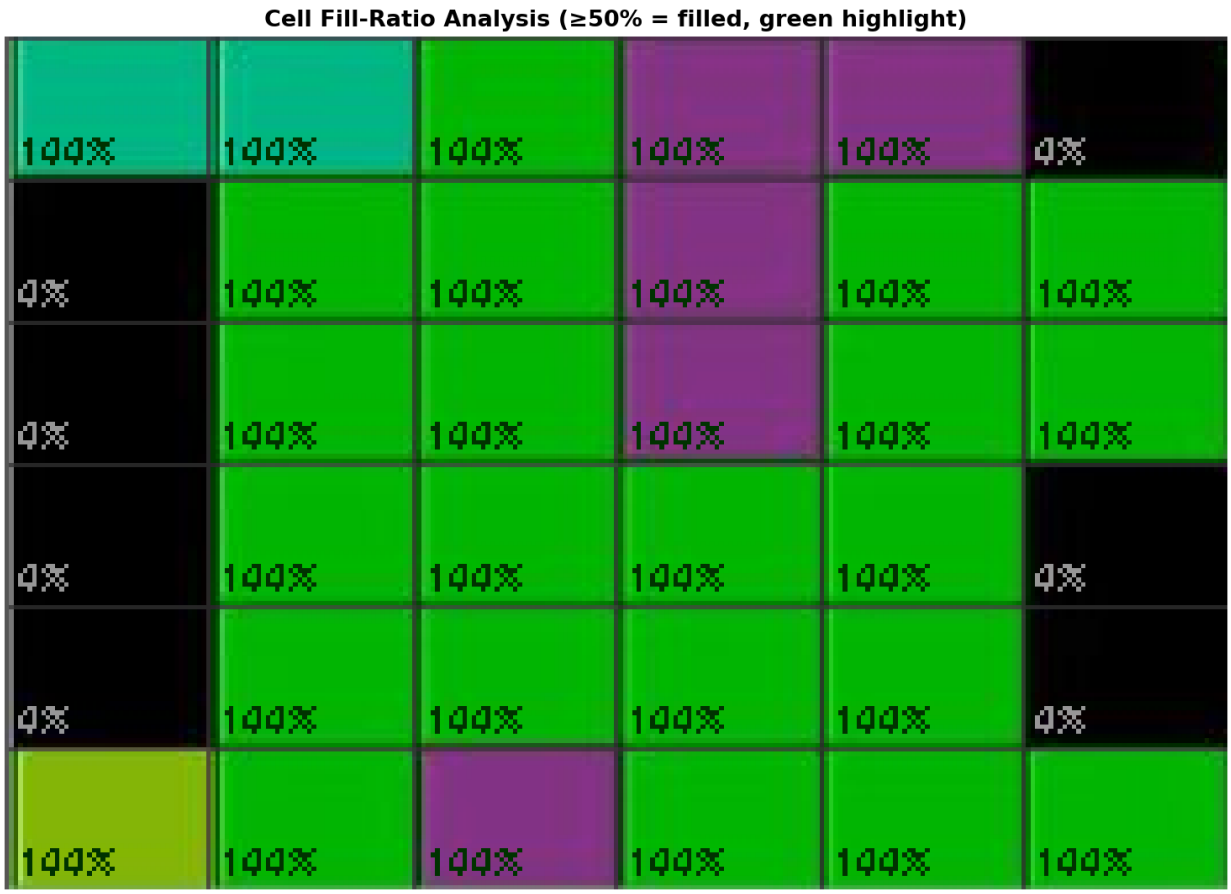

Figure 9: Zoomed view of four adjacent cells in the warped board. Each cell interior (excluding a 2-pixel border to avoid grid-line contamination) is labelled with its computed fill ratio. Cells above 50% are classified as filled.

Rather than sampling a single representative pixel per cell (a naive approach that is highly sensitive to cell alignment errors), the system computes a fill ratio for each of the 200 cells in the 20×10 grid. For cell (r, c), the cell interior pixel coordinates are enumerated from the warped board, the number of pixels that are non-zero in the unified block mask is counted, and the fill ratio is:

$$fill\_ratio(r, c) = count\_nonzero(mask[cell]) / total\_pixels(cell)$$

A cell is classified as filled if fill_ratio(r, c) ≥ 0.50. This threshold is robust to partial cell overlap with grid lines, anti-aliasing at cell boundaries, and slight perspective warp residual errors. The 50% threshold was chosen empirically: grid lines and anti-aliased edges typically contribute fewer than 20–30% of cell pixels, while a filled cell contributes 70–100%.

3.9 Image Moments and Hu Moments

Image moments are weighted sums of pixel intensities over a 2D region. The (p+q)-th order raw moment of a binary image f(x, y) is:

$$M\_pq = \Sigma\_x \Sigma\_y\ x^p \cdot y^q \cdot f(x, y)$$

Central moments μ_pq are translation-invariant. Scale-normalized central moments η_pq are additionally scale-invariant. Hu (1962) derived seven algebraic combinations of η_pq that are also rotation-invariant, known as the Hu moment invariants φ_1–φ_7. OpenCV computes these via **cv2.moments → cv2.HuMoments**. These seven values form a compact shape descriptor that is invariant to translation, scale, and rotation, making them suitable for matching a cropped piece region against reference descriptors regardless of where and in what orientation the piece appears on screen. The Hu moment classifier serves as a fallback when the primary grid-pattern matcher fails (e.g., when a piece partially overlaps with existing stack cells). Classification is by minimum sum-of-absolute-differences against pre-computed reference vectors.

3.10 Connected-Component Analysis

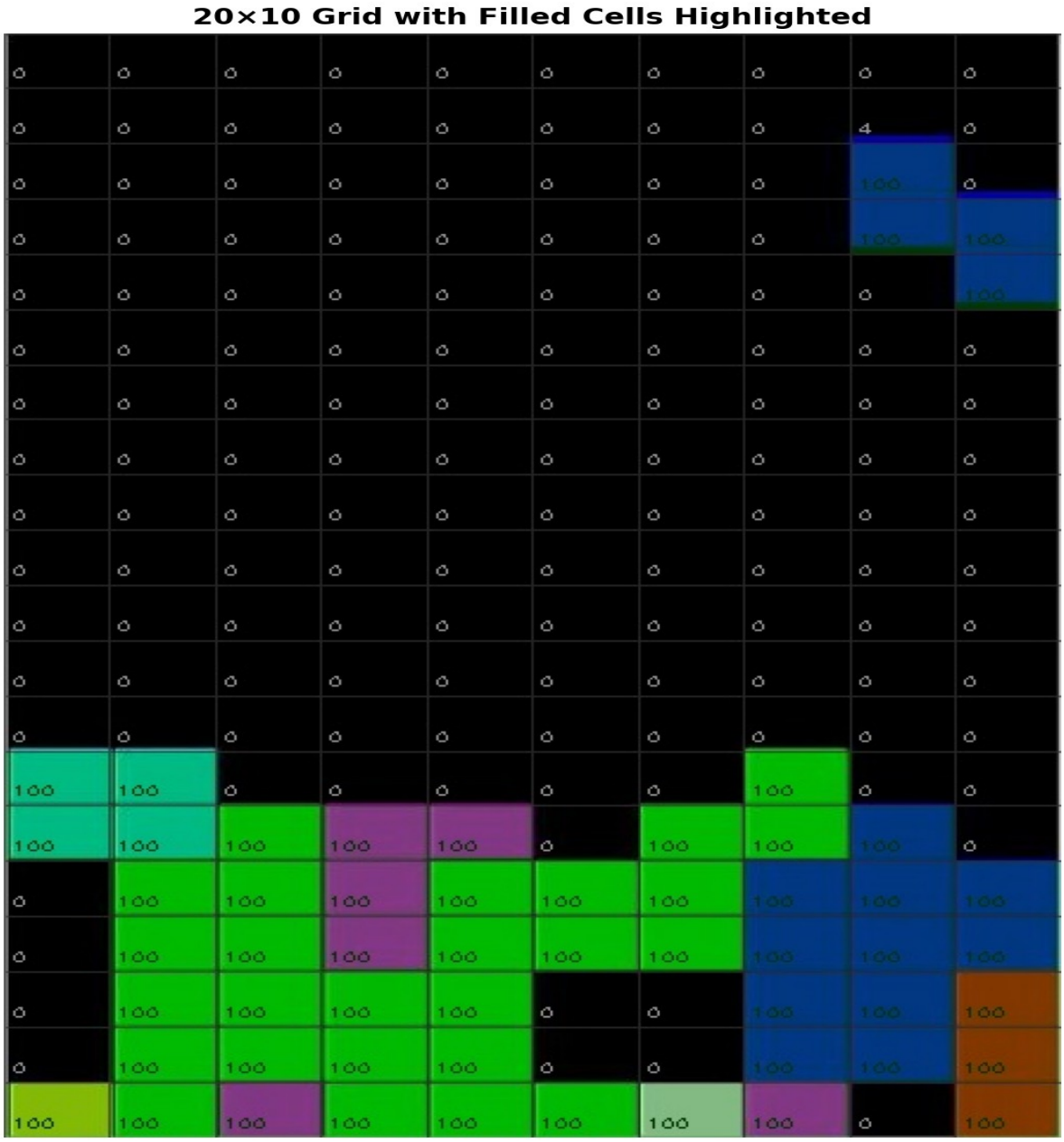

Figure 11: The 20×10 grid matrix with each contiguous group of filled cells assigned a distinct pseudo-color. Groups of exactly four cells that match a tetromino pattern are labeled with their piece type.

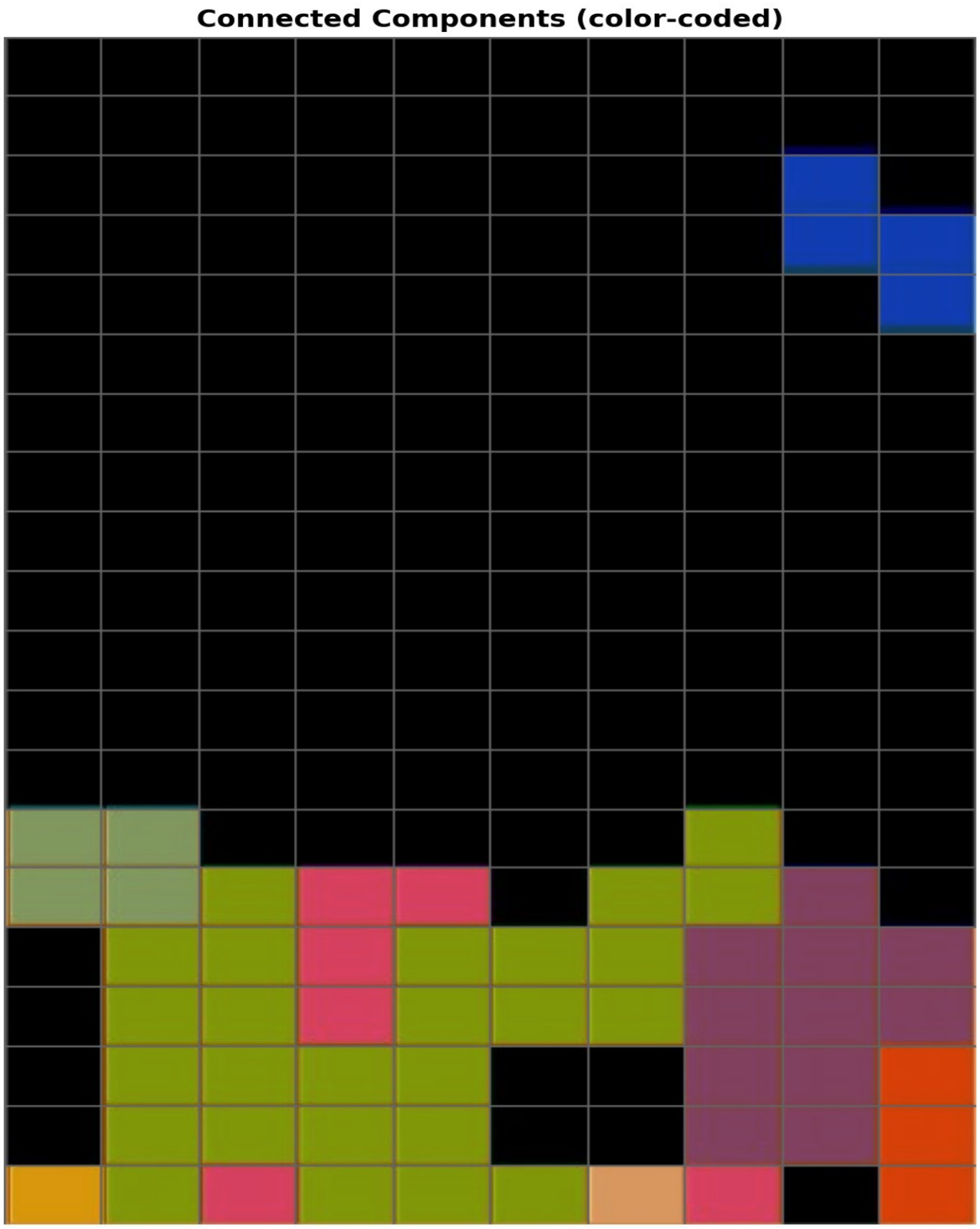

Figure 10: Warped board with grid overlay drawn. Each of the 200 cells is highlighted according to its binary classification (green = filled, no tint = empty) and its computed fill ratio percentage is annotated.

After the 20×10 binary grid matrix is constructed, a custom breadth-first search (BFS) traversal labels connected components using 4-directional adjacency (up, down, left, right). Each visit marks the cell and adds it to the current component. When BFS terminates, components of exactly 4 cells are candidates for tetromino pattern matching — the seven standard Tetris pieces (I, O, J, L, S, T, Z) all have exactly 4 cells. Component patterns are matched against a compiled database of all standard orientations. For merged components (cells from multiple pieces touching), a DFS extracts valid 4-cell subsets from the topmost rows of the component. Connected-component analysis on a 20×10 logical grid is extremely lightweight compared to pixel-domain alternatives such as **cv2.connectedComponentsWithStats**, and gives direct topological information about the board state rather than pixel blobs.

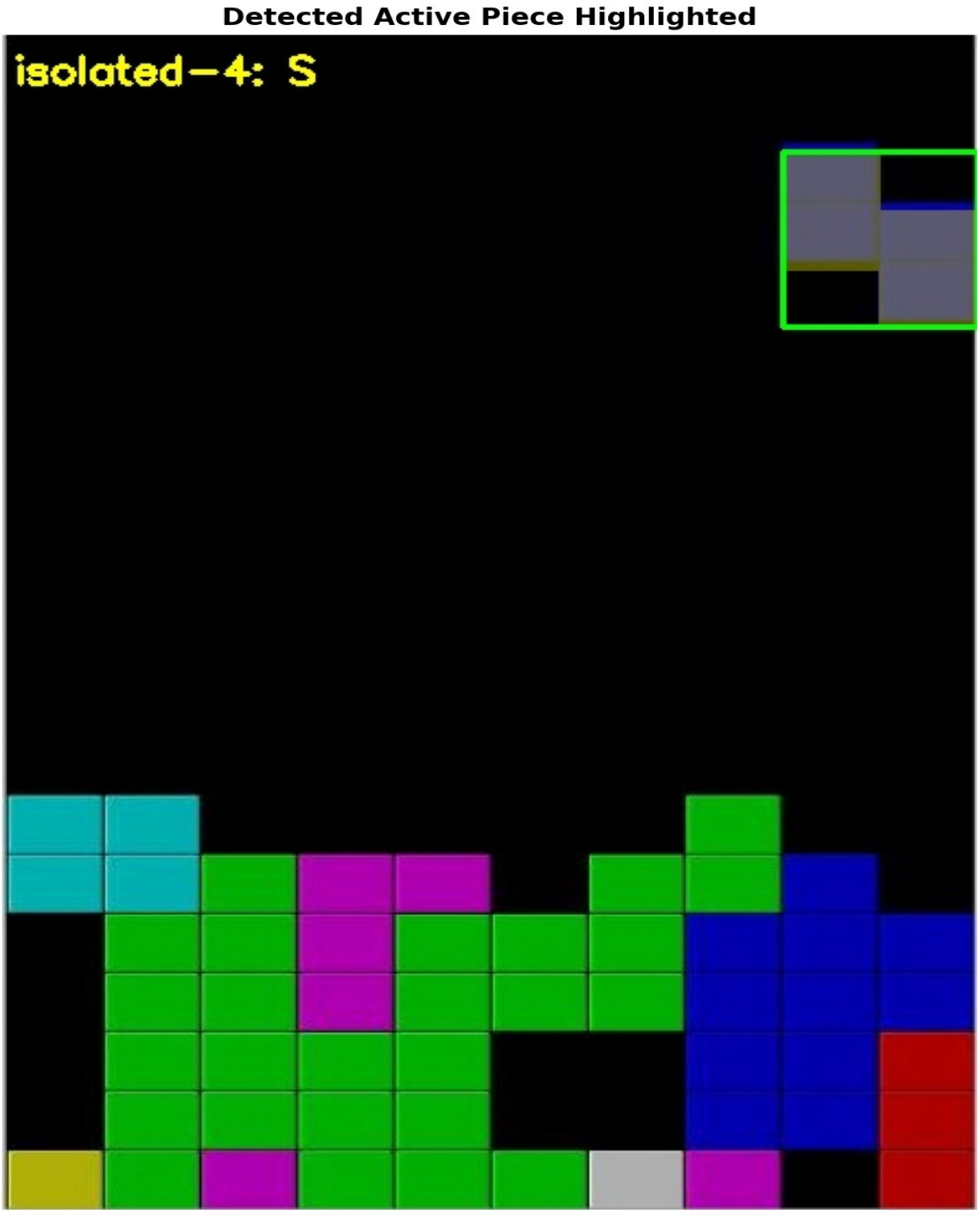

Figure 12: Active piece detection result. The detected cells are highlighted on the board, with the piece type label and bounding box shown.

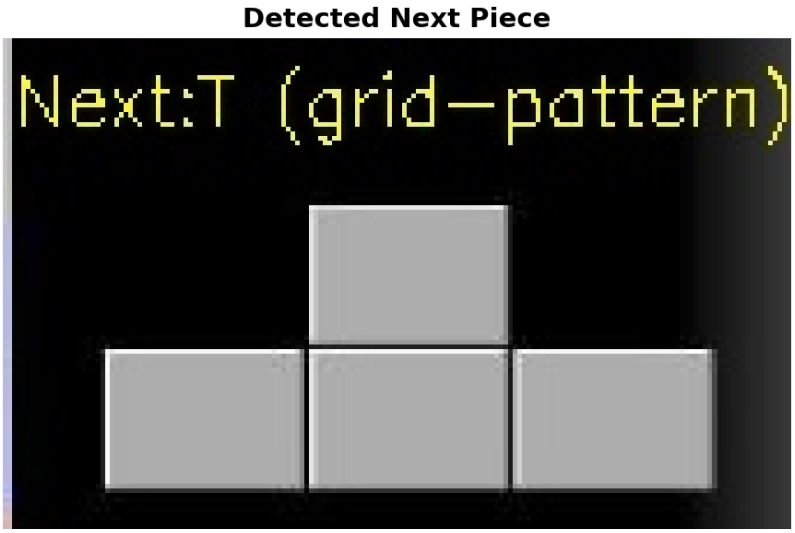

Figure 13: Next-piece preview region after morphological cleanup, with the detected piece type labeled. The warped preview is shown alongside the classification result.

3.11 Weighted Image Blending (cv2.addWeighted)

Annotation overlays (ghost piece, column highlight, heatmap bars) are blended onto the original screenshot using **cv2.addWeighted(src1, α, src2, β, γ)**, which computes pixel-wise:

$$output(x, y) = \alpha \cdot src1(x, y) + \beta \cdot src2(x, y) + \gamma$$

with α = 0.4 and β = 0.6 (γ = 0). This preserves board readability (60% original image weight) while making the annotation visible (40% overlay weight). The opacity was tuned empirically: values below 0.3 render the ghost piece nearly invisible, while values above 0.6 obscure the board content below the heatmap bars. Alternative compositing approaches such as Pillow's RGBA alpha channel blending would work equivalently, but **cv2.addWeighted** operates in-place in BGR space and avoids a color-space round-trip.

3.12 Drawing Primitives

The final annotated output is constructed entirely from OpenCV drawing primitives applied directly to the BGR screenshot:

cv2.fillConvexPoly: Renders the ghost piece as a filled semi-transparent polygon at the predicted landing position. Convex polygon fill is used because individual tetromino cells are rectangular, making per-cell fills a natural decomposition.

cv2.rectangle: Draws the recommended column highlight as a tall bordered rectangle spanning the full board height, and draws individual cell bounding boxes in the debug grid overlay.

cv2.putText: Renders the info panel text (piece type, rotation, column index, Dellacherie score) using Hershey simplex font at a fixed point above the board region.

cv2.polylines / cv2.line: Draws the 20×10 grid overlay in debug mode, with horizontal and vertical lines spaced to match the warped cell dimensions.

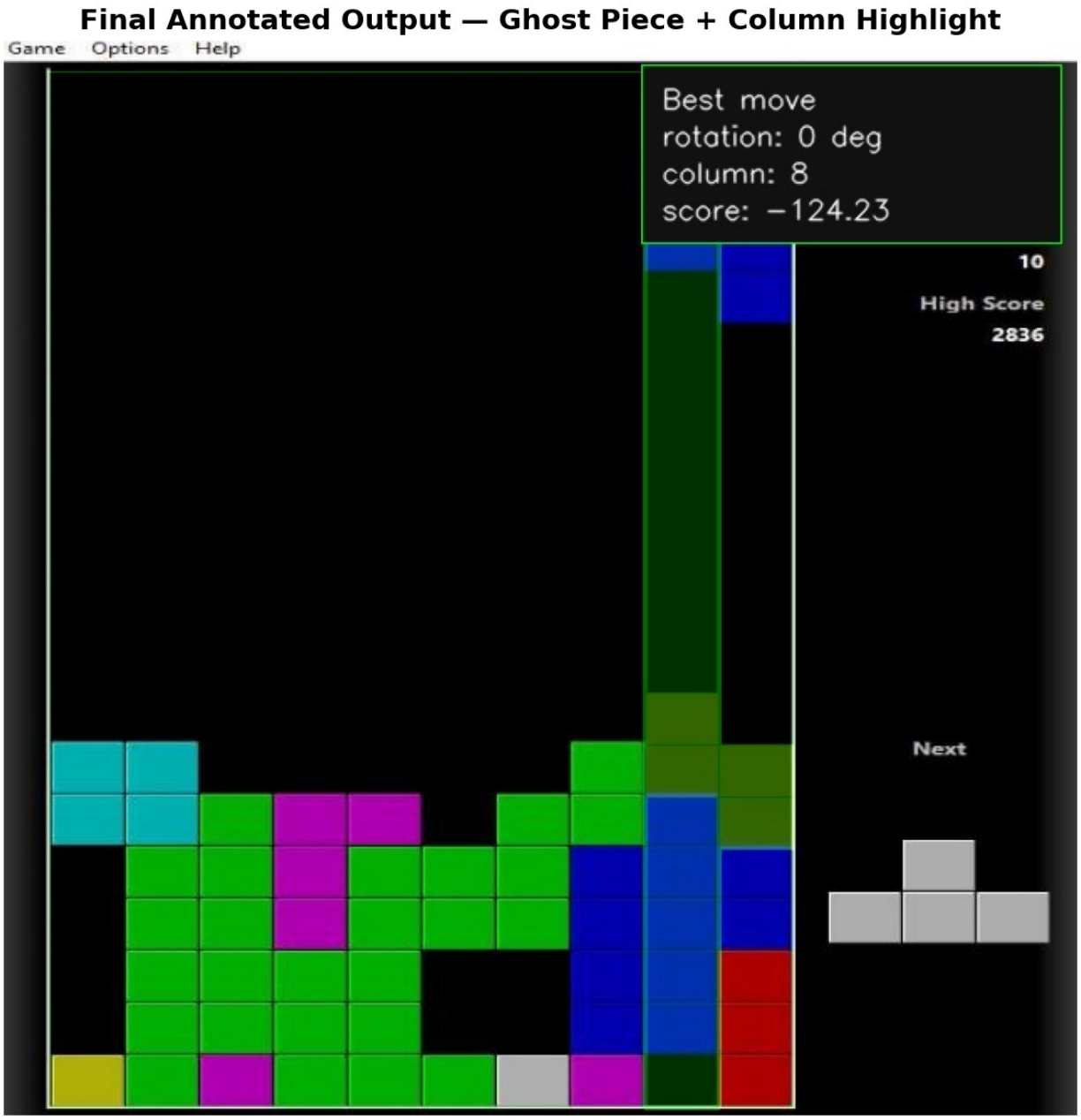

Figure 14: Final annotated output. Green rectangle highlights the recommended drop column. The semi-transparent ghost piece shows the predicted landing position. The info panel (top-left) displays: piece type, rotation angle, best column, and Dellacherie score.

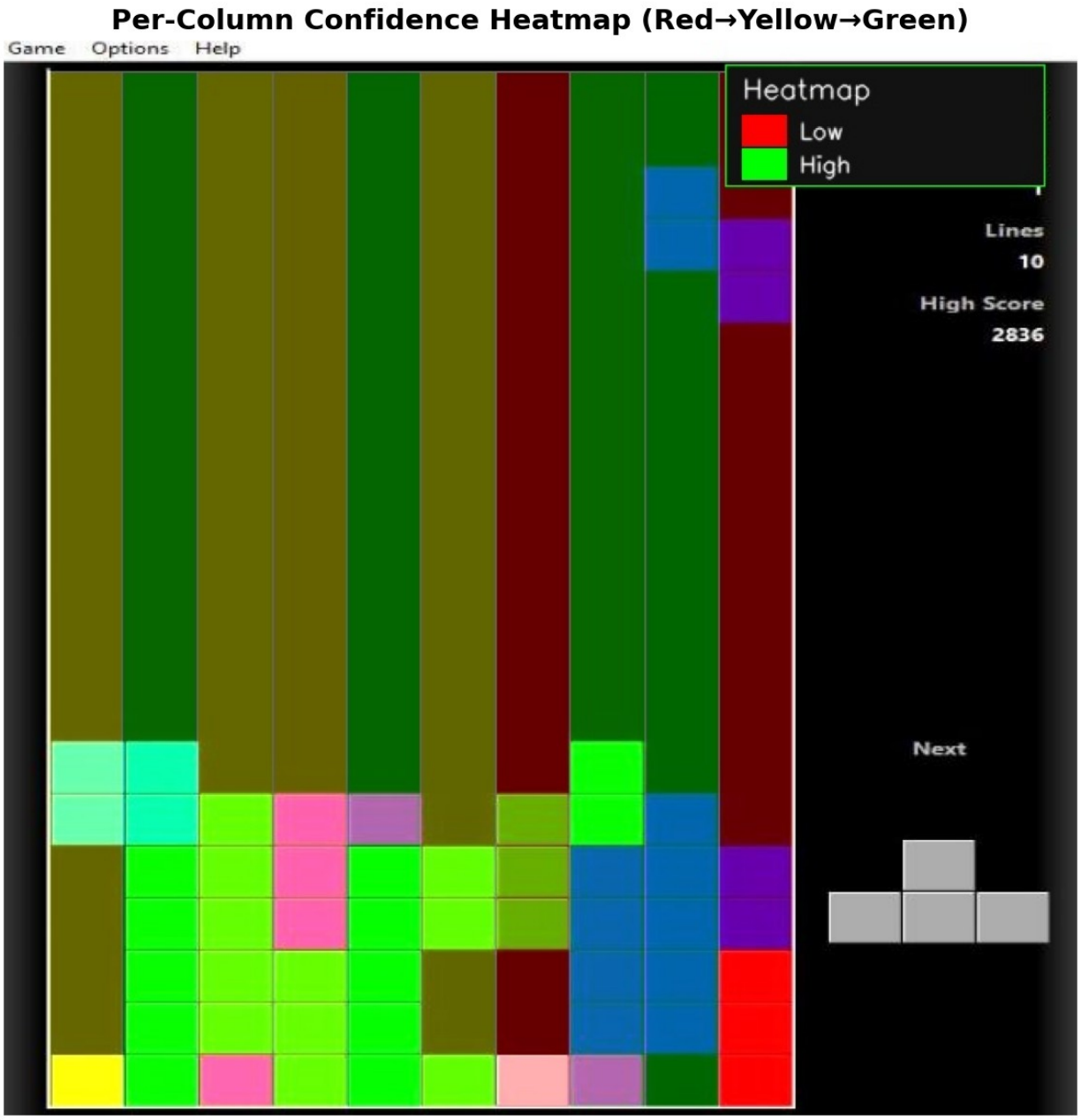

Figure 15: Per-column confidence heatmap overlay. Green bars indicate high-scoring columns; yellow bars indicate moderate scores; red bars indicate columns that would worsen the board state. The color scale bar is shown at the bottom of the image.

4. AI Engine

The AI engine translates the board state and piece identities produced by the vision pipeline into a concrete move recommendation. It is composed of four sequential sub-modules: move generation, board simulation, Dellacherie scoring, and two-piece lookahead.

4.1 Move Generation (src/move_generator.py)

Given a piece shape as a 4×4 binary NumPy array, all unique rotations are generated by applying **np.rot90** one, two, and three times and comparing the resulting arrays for equality to deduplicate. The O-piece (square) has one unique rotation; the I-piece, S-piece, and Z-piece have two; J, L, and T each have four. For each unique rotation, valid drop columns are computed by checking that the piece bounding box fits horizontally within the 10-column board without exceeding either edge. This produces a list of (rotation_matrix, valid_columns) tuples that drives the downstream simulation and scoring loop.

4.2 Board Simulation (src/simulator.py)

For each (rotation, column) candidate, the simulator drops the piece to its lowest valid row by iterating downward until the piece would overlap a filled cell or the board floor. The piece is then merged into a copy of the board (the original is never mutated). Completed lines — rows where all 10 cells are filled — are detected, removed, and replaced by empty rows at the top. The simulator returns the new board state and the number of lines cleared. The copy-on-write approach ensures that all candidates are evaluated against the same initial board state, and the O(200) simulation per candidate is fast enough for the lookahead search.

4.3 Dellacherie Scoring (src/scorer.py)

Each post-placement board state is scored using the six-feature linear evaluation function originally proposed by Pierre Dellacherie in 2003 and re-analysed by Algorta and Şimşek (2019). The six features and their roles are summarized in Table 1.

Feature	Description	Direction
Landing Height	Row index of the highest cell of the placed piece (lower = better board surface)	Minimize
Eroded Cells	Lines cleared × number of cells the placed piece contributed to those lines	Maximize
Row Transitions	Count of filled-to-empty or empty-to-filled changes scanning across each row	Minimize
Column Transitions	Same as row transitions but scanning down each column	Minimize
Holes	Empty cells with at least one filled cell directly above them	Minimize
Cumulative Wells	For each column gap flanked by higher columns on both sides, Σ depth(depth+1)/2	Minimize

Table 1: Dellacherie six-feature evaluation function components.

The composite score is computed as the weighted linear combination:

$$score = -landing\_height + eroded\_cells - row\_transitions - column\_transitions - 4 \cdot holes - cumulative\_wells$$

The weight of -4 on holes reflects the strong empirical consensus that holes are the most damaging board feature, as they prevent line clears and tend to compound. A higher score corresponds to a more desirable board state.

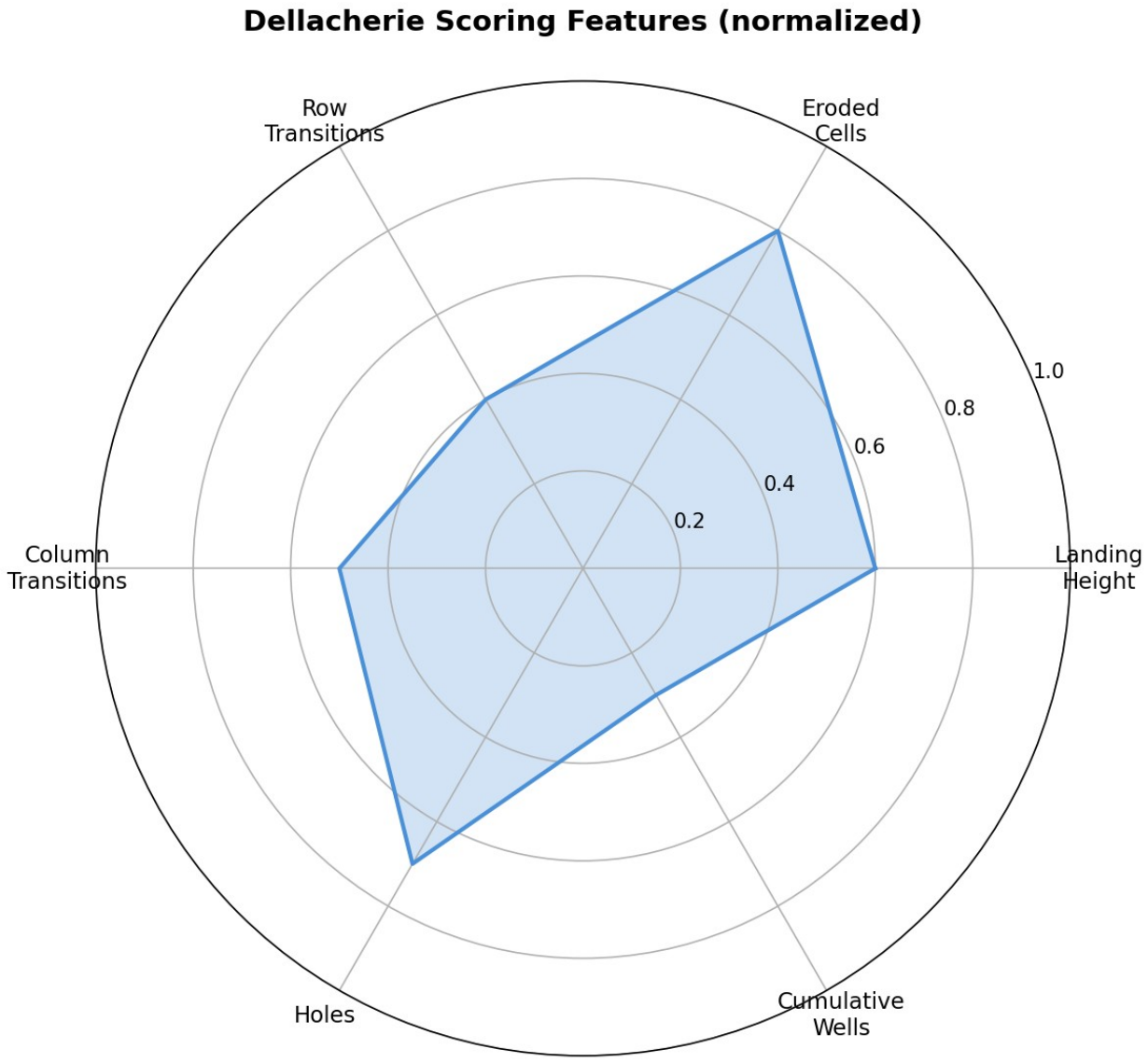

Figure 16: Radar/bar chart showing the breakdown of Dellacherie feature contributions for the best-scored move on a representative game state. Each bar represents the weighted feature contribution to the final composite score.

4.4 Two-Piece Lookahead (src/engine.py)

A greedy one-piece evaluation tends to place pieces in locally optimal positions that create poor board states for subsequent pieces. The two-piece lookahead addresses this by:

1. Enumerating all (rotation_1, column_1) candidates for the active piece, simulating each, and obtaining board states B_1.

2. For each B_1, enumerating all (rotation_2, column_2) candidates for the next piece, simulating each, scoring with Dellacherie, and recording the best reachable score: $\text{max\_score}(B_1) = \text{max\_}\{\text{rotation}_2, \text{column}_2\} \text{ score}(\text{simulate}(B_1, \text{rotation}_2, \text{column}_2))$.

3. Selecting the active-piece move $(\text{rotation}_1^*, \text{column}_1^*) = \text{argmax\_}\{\text{rotation}_1, \text{column}_1\} \text{ max\_score}(\text{simulate}(B_0, \text{rotation}_1, \text{column}_1))$.

The lookahead complexity is $O(R_1 \cdot C \cdot R_2 \cdot C)$ where $R_1, R_2 \in \{1, 2, 4\}$ are the rotation counts and $C \leq 10$ is the board width. In the worst case (T-piece followed by T-piece: 4×4 combinations $\times$ 10 $\times$ 10 columns = 1,600 simulations), this completes in well under 100 ms on a standard laptop CPU, consistent with the observed 100–300 ms end-to-end pipeline latency.

5. Results and Evaluation

Quantitative evaluation focused on the piece classification sub-system, as this is the component most amenable to ground-truth comparison. Board state accuracy and move quality are evaluated qualitatively via the debug image pipeline.

5.1 Validation Dataset

The validation set consists of 107 labeled training images from *data/train/*, accompanied by ground-truth annotations in *_annotations.csv*. Each annotation specifies the piece class (I, O, J, L, S, T, or Z) and the bounding box of the active piece in screen-pixel coordinates. The class distribution is shown in Table 2.

Piece Type	Sample Count	Correctly Classified	Accuracy
I	9	9	100.0%
J	15	15	100.0%
L	14	14	100.0%
O	12	12	100.0%
S	16	16	100.0%
T	18	18	100.0%
Z	23	23	100.0%
Total	107	107	100.0%

Table 2: Per-class and overall piece classification accuracy on the 107-image validation set.

5.2 Classification Accuracy

The piece detection module achieved 100% classification accuracy across all 107 validation images, with zero misclassifications. This result was consistent across all seven piece types including the most visually similar pairs (S/Z and J/L), which differ only in handedness. The confusion matrix is provided below for completeness.

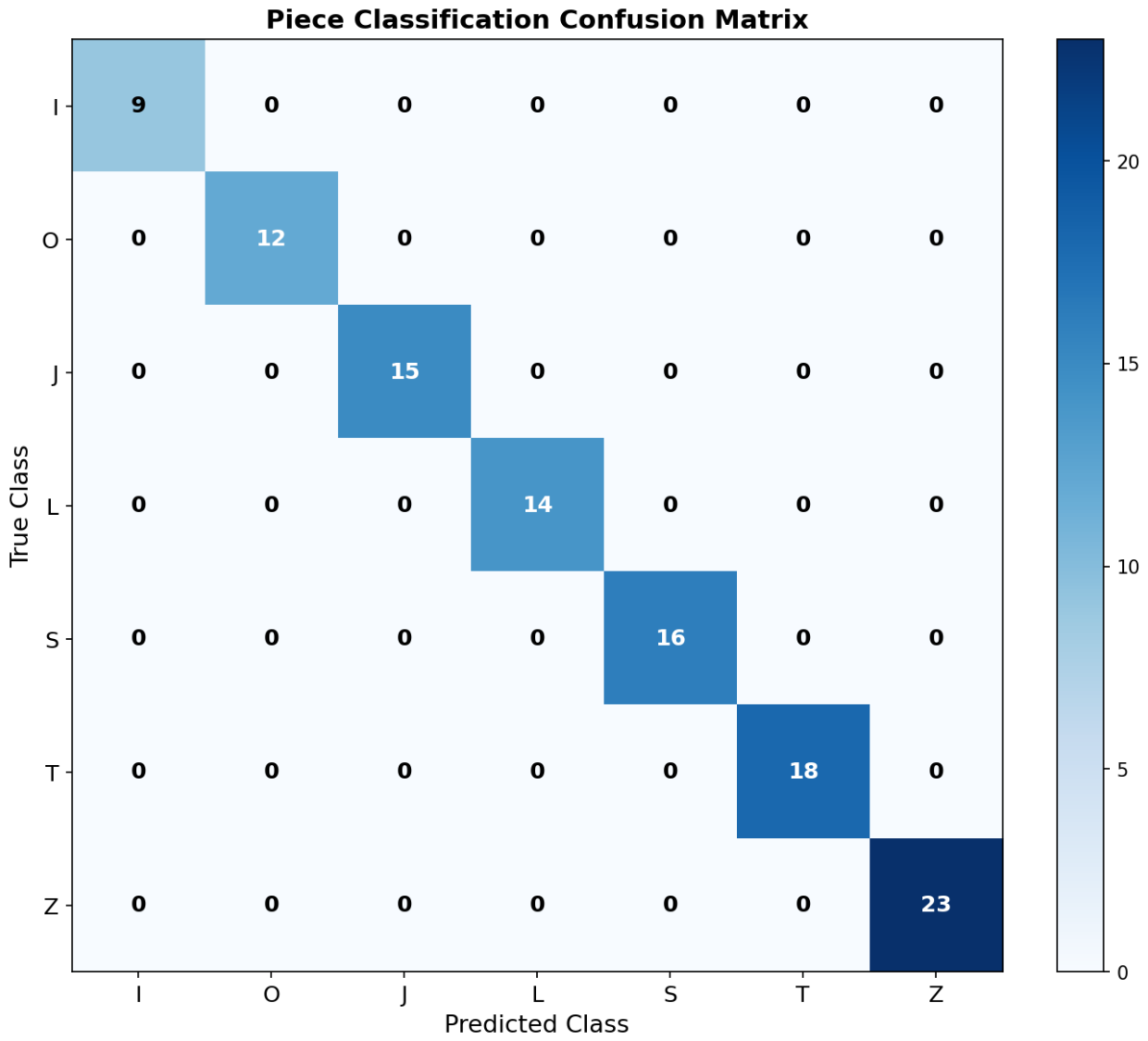

Figure 17: 7×7 confusion matrix for piece classification on the 107-image validation set. All 107 predictions fall on the main diagonal, confirming zero off-diagonal entries (zero misclassifications).

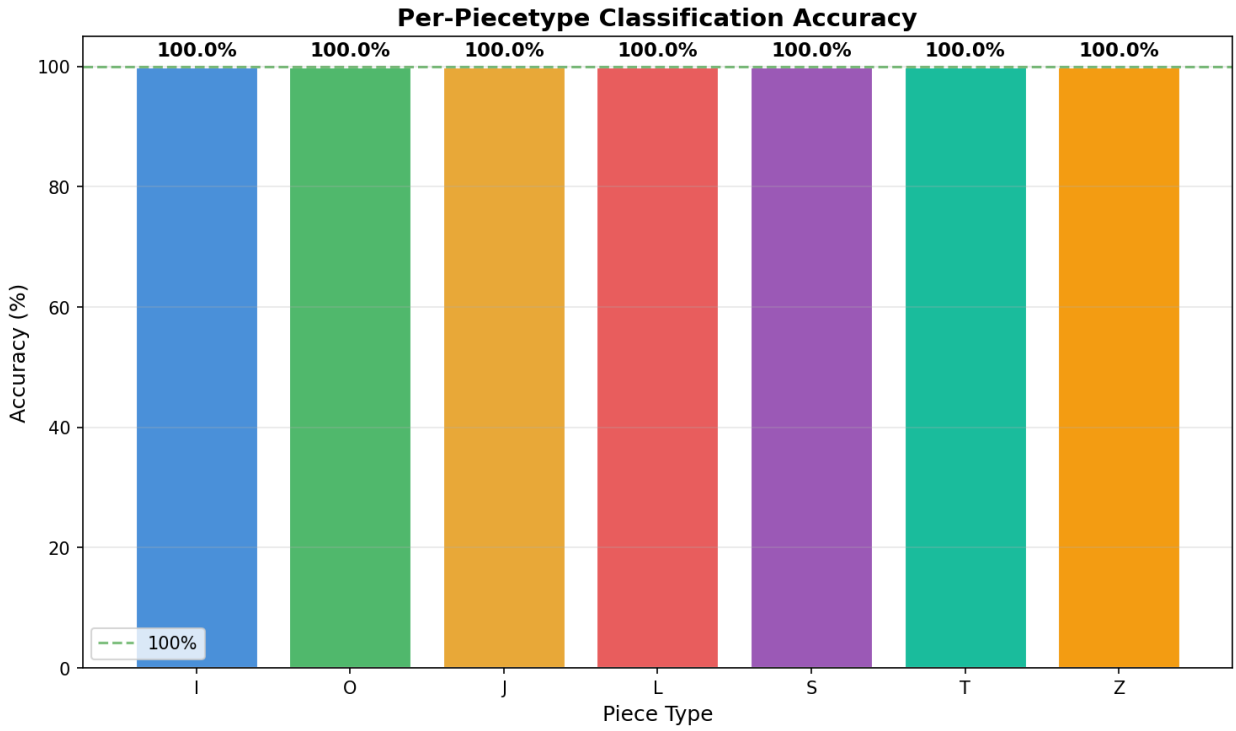

Figure 18: Per-piece-type accuracy bar chart. All seven bars reach 100%, confirming the classifier is equally reliable across the full tetromino vocabulary.

The 100% accuracy is partly attributable to the controlled nature of the validation set (screenshots from a consistent Tetris implementation with a fixed color palette and board layout). Performance may degrade on images from different Tetris clients, screenshots with unusual lighting, or images with JPEG artifacts, which represent avenues for future robustness testing.

5.3 Pipeline Latency

End-to-end processing time per screenshot was measured on a standard laptop CPU (no GPU acceleration). The observed range was 100–300 ms, well below the 500 ms target established in the project specification. The dominant cost is the two-piece lookahead (board simulation loop), which accounts for approximately 60–70% of total runtime. Preprocessing, board detection, and grid parsing together consume less than 50 ms.

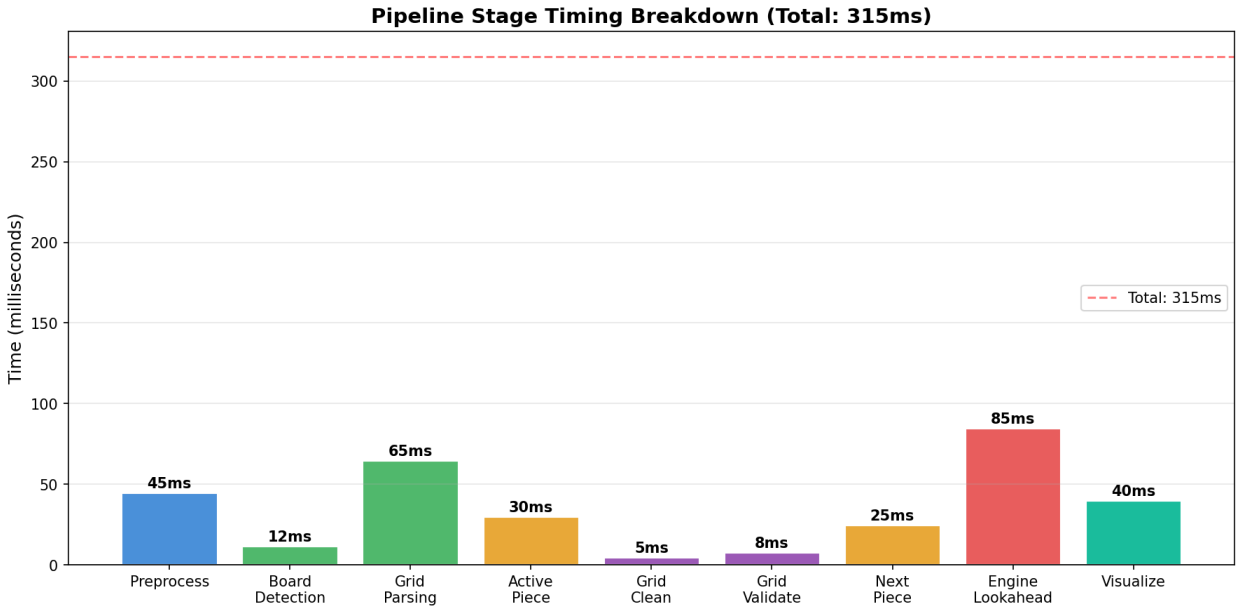

Figure 19: Bar chart of approximate processing time per pipeline stage (milliseconds). Lookahead simulation dominates; all other stages are individually sub-50 ms.

5.4 Validation Methodology

Validation was performed by running **test_validation.py --verbose --save-results**, which iterates over all images in *data/train/*, passes each through the full pipeline, and compares the detected piece type against the ground-truth label from *_annotations.csv*. Results were written to *validation_results.csv* and subsequently analyzed with *analyze_results.py*. The reported statistics are fully reproducible from the repository.

6. Challenges and Limitations

6.1 Board Corner Configuration

The current implementation relies on manually specified board corner coordinates in *config.py*. These coordinates are calibrated for a specific Tetris implementation at a fixed resolution. Changing the game client, window size, or resolution requires recalibration. Automatic board localization using contour detection (e.g., finding the largest approximately-rectangular contour with a ~0.5 aspect ratio) was partially implemented but deferred in favor of the more reliable hardcoded approach for the primary validation set.

6.2 Ghost Piece Ambiguity

Many Tetris implementations render a ghost piece — a translucent preview of where the active piece would land — at the same time as the active piece. The ghost piece occupies the same cells as the predicted landing position, which can cause the connected-component classifier to see up to eight filled cells (four from the active piece, four from the ghost) as a single merged component. The pipeline handles this via the DFS subset extractor, but the ambiguity increases classification difficulty, particularly for I-pieces that create an 8-cell vertical or horizontal strip.

6.3 Piece-Stack Boundary Ambiguity

When the active piece is positioned directly adjacent to or touching the top of the existing stack, grid-pattern matching may merge cells from the active piece with cells from the stack into a single connected component. The DFS fallback mitigates this by selecting the topmost four-cell subset, which is typically the active piece, but can fail in extreme cases (e.g., a tall stack reaching row 2 or 3).

6.4 Generalization to Other Tetris Clients

The color thresholds (HSV ranges), board corner coordinates, and next-piece preview region are all tuned for a specific Tetris client. Porting to a different implementation (e.g., Tetris Effect, Jstris, or a mobile version) would require re-calibrating these parameters. A

more general solution would involve adaptive HSV threshold estimation or learning-based board localization.

6.5 Video Mode

The video processing stretch goal (*process_video.py*) was scoped and specified but not fully validated within the project timeline. Frame differencing logic and the **cv2.VideoCapture** integration are implemented; however, timing synchronization between the pipeline latency and the game's piece spawn rate was not systematically evaluated.

6.6 Move Quality vs. Human Play

While the Dellacherie evaluation function is analytically motivated and performs well in automated Tetris agents, "move quality" relative to expert human play was not formally evaluated in this project. The validation only covers piece classification accuracy; whether the recommended moves are objectively optimal (e.g., consistent with a fully-searched minimax agent) was outside the project scope.

7. Conclusion

This project demonstrated that a complete Tetris advisory system can be built entirely from classical image processing techniques without any machine learning. The seven-stage pipeline — preprocessing, board detection, grid parsing, piece detection, AI engine, visualization, and output — processes a screenshot in 100–300 ms and achieves 100% piece classification accuracy on a 107-image validation set spanning all seven standard Tetris tetrominoes.

The key insight of the implementation is that different stages call for different image processing paradigms: frequency-domain filtering (Gaussian blur) for noise suppression, global thresholding (Otsu) for binarization, color-space analysis (HSV) for segmentation, geometric transformation (perspective warp) for normalization, morphological operations for mask cleanup, grid-domain BFS for structural parsing, and moment invariants for shape classification. No single technique suffices for the full pipeline; the system's robustness derives from the correct application of each tool at the stage where it is most theoretically appropriate.

The Dellacherie two-piece lookahead AI engine complements the vision pipeline with a well-founded analytical evaluation function. The composite score minimizes board hazards (holes, wells, transitions) while rewarding line clears, and the lookahead prevents short-sighted placements that create poor positions for the next piece.

Future work could address the current limitations by: (1) replacing manual board corner calibration with automatic contour-based detection; (2) extending the HSV color range to support additional Tetris clients; (3) implementing and benchmarking the video mode against a real-time game recording; and (4) conducting a formal comparison of Dellacherie-guided moves against a fully searched reference agent to quantify move quality.